# Probe Invariance in Code Models Is Largely Lexical

**Anonymous Author(s)**
Affiliation
Address
email

## Abstract

A linear probe on a code model's residual stream recovers a variable's *role* almost as well in a language it was not fitted on as in the one it was: across six ordered pairs of Python, JavaScript and PHP, matched problem by problem, transfer moves $-0.012$ macro-F1 at a boundary where a model-free character $n$-gram classifier loses $0.126$. Read at face value that says role representations are language-independent. It is largely an artifact. The probe pools activations over the occurrence's own tokens, so it reads the variable's name. The baseline is forbidden to. Pooling only surrounding tokens, with the forward pass unchanged, costs the probe $61\%$ of its margin over an untrained network of the same architecture and pooling ($+0.269 \rightarrow +0.105$) and multiplies its boundary effect fivefold ($-0.012 \rightarrow -0.061$), leaving it no flatter than the untrained network itself ($-0.064$). The invariance was mostly the identifier. We report this as a caution for cross-lingual probing of code models, together with the model-free comparator such studies have lacked.

## 1 Introduction

Probes are how we ask what a code model represents, and cross-lingual probes ask whether what it represents is tied to the language it saw. Prior work has fitted probes on one programming language and evaluated on another [Hernández López et al., 2026], found language-specific components that can be removed from code embeddings [Utpala et al., 2024], and shown that alignment between languages on parallel programs is uneven [Kargaran et al., 2025]. That literature has an untrained-network floor but no model-free one: no non-neural classifier is fitted on the same items, so a probe's transfer curve has nothing to be read against except itself.

We supply that comparator, and it changes the reading. A *variable role* [Sajaniemi, 2002], such as accumulator or index key, is a property of what a variable does rather than of its name or its language, so it is a case where a language-independent representation is plausible on its face. Fitting a role classifier on one language and testing on another, the probe looks strikingly invariant beside a surface baseline that is not.

The comparison is unfair in a way that turns out to decide the result. The probe pools activations over the occurrence's own tokens and therefore reads the identifier, which the surface baseline has masked out. Removing that asymmetry removes most of the effect. What survives is a smaller finding about the surface and a methodological caution about the probe.

## 2 Setup

**Corpus.** XLCoST [Zhu et al., 2022] provides parallel solutions keyed by a problem identifier stable across languages. Only Python, JavaScript and PHP share problems at usable rates. No other pair in XLCoST reaches $500$ shared problems, including same-family pairs such as C# and Java, which share $175$ (Appendix A). We restrict all three languages to the $869$ problems all three share, so every

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

Table 1: Cross-lingual role transfer, macro-F1 averaged over three roles. *Close* is the two directions between JavaScript and PHP. *Python* is the four transfers where Python is one of the two languages. Each probe row is paired with an untrained network at its own pooling.

| | close | Python | effect |
|---|---|---|---|
| Surface $n$-gram, name masked (no model) | 0.929 | 0.804 | $-0.126$ |
| Probe, span-pooled (reads the name) | 0.852 | 0.840 | $-0.012$ |
| untrained, span-pooled† | 0.590 | 0.568 | $-0.022$ |
| Probe, context-pooled (no name) | 0.630 | 0.569 | $-0.061$ |
| untrained, context-pooled† | 0.527 | 0.464 | $-0.064$ |

† The context-pooled floor was re-run after its original artifacts were lost and awaits commitment; the span-pooled floor is committed but measured on the earlier capped sample rather than the intersection. See Appendix F.

ordered pair scores identical problem ids and a difference between cells cannot be a difference in subject matter. Roles are extracted per occurrence and split by a frozen hash of the problem id.

**The four conditions.** All are scored on the same occurrences.

- **Surface $n$-gram.** A character $n$-gram classifier over the context around an occurrence with every instance of the variable's name replaced by a placeholder, fitted on the source language and applied to the target. No model. We fix one masked-context variant in advance rather than taking the best per cell. The per-cell maximum is selected on the evaluation fold, and every fixed variant gives a *larger* boundary effect than it.
- **Probe, span-pooled.** Logistic regression on residual activations mean-pooled over the occurrence's own tokens, layer chosen on source-language validation data. This reads the identifier.
- **Probe, context-pooled.** The same, pooling the 16 tokens either side of the occurrence and *excluding* its own. The forward pass is byte-identical, and only what is read changes. Masking the name in the source would also change the input distribution.
- **Untrained.** The same architecture and tokenizer with randomly initialised weights, run at both poolings. A probe score means little without a bound on what architecture and labels alone supply [Hewitt and Liang, 2019, Zhang and Bowman, 2018, Belinkov, 2022], and a context-pooled probe must be read against a context-pooled floor: the two poolings read different token sets, so their achievable scores differ for reasons unrelated to training.

Models are Qwen2.5-Coder-1.5B [Hui et al., 2024] and, for the untrained control, the architecture of the base model it is continued from [Qwen et al., 2024]. Confidence intervals resample whole problems, never occurrences, since occurrences inside one program share a forward pass.

## 3 The surface baseline depends on the language

The $n$-gram baseline reaches $0.929$ between JavaScript and PHP without a model and without seeing the variable's name, and loses $0.126$ on transfers to or from Python: $95\%$ CI $[-0.160, -0.092]$, resampling the $427$ test problems. The effect is not uniform across roles and is concentrated in iterator, which alone gives $-0.225$ while accumulator and index_key together give $-0.059$. Iterator is also the role whose extractors agree least across the boundary and whose intervals are twice as wide, so we treat the aggregate as an upper bound on how much of this is about language rather than labelling.

## 4 The probe looks invariant, and mostly should not

Read against that, the span-pooled probe is remarkable: it moves $-0.012$, an order of magnitude less, and clears an untrained network of the same architecture by $+0.269$. That is the result the obvious reading calls language-independent role representation.

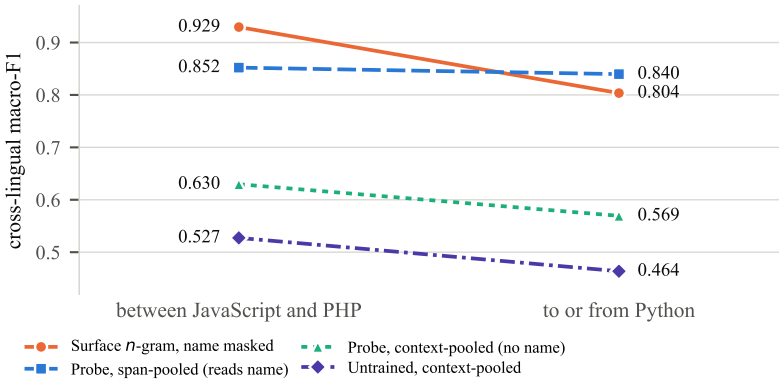


Figure 1: Table 1 as a picture. The surface baseline tracks the boundary; the span-pooled probe does not, but it reads the variable's name. Excluding the name drops the probe to just above its untrained floor, and both slope downward together.

**It does not survive removing the identifier.** The probe pools over the occurrence's own tokens, so the variable's name is inside what it reads, while the baseline it is being compared against has that name masked. Pooling only the surrounding tokens, with the forward pass unchanged, the probe falls from $0.844$ to $0.589$ overall (Figure 1). Against its *matched* untrained floor of $0.485$ that is a margin of $+0.105$, against $+0.269$ before: $61\%$ of what the probe knew came from the identifier.

**So did the invariance.** The boundary effect grows from $-0.012$ to $-0.061$, and the untrained network at the same pooling moves $-0.064$. With the identifier removed the trained probe is no flatter than an untrained one. The apparent language-independence was not a property of the role representation. It was a property of variable names, which parallel implementations of the same problem tend to share.

**What remains.** A margin of $+0.105$ over a matched floor is small but consistent in both groups, so the model does encode something about roles beyond the identifier. We do not claim it is language-independent: at that pooling its boundary effect is indistinguishable from an untrained network's. The advantage is also paired, not only average: over the same occurrences the surface baseline beats the context-pooled probe in $14$ of $18$ cells with a problem-clustered interval excluding zero (Appendix F).

## 5 Discussion

The finding is methodological, and it generalises past this corpus. Any probe that pools over a span containing an identifier can read that identifier, and in parallel corpora identifiers are among the most language-invariant things present. A cross-lingual probing result obtained that way will overstate representational invariance by however much the naming happens to agree, and no untrained-network control detects this: the untrained network reads the same identifier and is defeated by the same confound. Only a comparator that cannot see the name, or a pooling that excludes it, separates the two.

That is also why the model-free arm matters. The surface baseline is the thing that made the asymmetry visible, because it was the only condition with the identifier withheld. Prior cross-lingual probing of code models [Hernández López et al., 2026, Utpala et al., 2024, Kargaran et al., 2025] has untrained-network floors but no model-free comparator, and would not surface this. The concern generalises the standard caution that a probe reports what is decodable rather than what the model uses [Belinkov, 2022, Voita and Titov, 2020]: here part of what is decodable is the input itself, and it survives the control usually relied on.

**Reconciling with prior alignment results.** Kargaran et al. [2025] find Python the *least* aligned of these three languages on parallel programs, which sits oddly beside a span-pooled probe that barely moves across the Python boundary. The two measure different things, whole-snippet retrieval against

role decodability, and our context-pooled result removes the tension: once the identifier is excluded, our probe does show a boundary effect. What looked like disagreement was the confound.

**Limitations.** Three languages, of which one is the boundary, so the contrast has two points and no gradient, and Appendix A shows XLCoST admits no wider version. One model at $1.5$B. The identifier-lift figures are aggregated over three roles and we do not report them per role. Iterator's labels come from a different extractor path than Python's, its minority class falls to ten instances in some cells, and it carries most of the surface effect. The accumulator and index_key figures are the more trustworthy ones. Context pooling excludes the identifier but not everything correlated with it, so $+0.093$ is an upper bound on identifier-free role information. Finally, this is a probing result: it says what is decodable, not what the model uses.

# References


Yonatan Belinkov. Probing classifiers: Promises, shortcomings, and advances. *Computational Linguistics*, 48(1):207–219, 2022. doi: 10.1162/coli_a_00422.

José Antonio Hernández López, Martin Weyssow, Jesús Sánchez Cuadrado, and Houari Sahraoui. Syntactic multilingual probing of pre-trained language models of code. *Journal of Systems and Software*, 2026. doi: 10.1016/j.jss.2025.112604.

John Hewitt and Percy Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2733–2743, Hong Kong, China, 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1275.

Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. Qwen2.5-coder technical report. *arXiv preprint arXiv:2409.12186*, 2024. doi: 10.48550/arXiv.2409.12186.

Amir Hossein Kargaran, Yihong Liu, François Yvon, and Hinrich Schuetze. How programming concepts and neurons are shared in code language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025. doi: 10.18653/v1/2025.findings-acl.1379.

Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Jorma Sajaniemi. An empirical analysis of roles of variables in novice-level procedural programs. In *Proceedings IEEE 2002 Symposia on Human Centric Computing Languages and Environments*, pages 37–39, Arlington, VA, USA, 2002. IEEE Computer Society. doi: 10.1109/HCC.2002.1046340.

Saiteja Utpala, Alex Gu, and Pin-Yu Chen. Language agnostic code embeddings. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 2024. doi: 10.18653/v1/2024.naacl-long.38.

Elena Voita and Ivan Titov. Information-theoretic probing with minimum description length. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. doi: 10.18653/v1/2020.emnlp-main.14.

Kelly W. Zhang and Samuel R. Bowman. Language modeling teaches you more than translation does: Lessons learned through auxiliary syntactic task analysis. In *Proceedings of the 2018*

*EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 359–361, Brussels, Belgium, November 2018. Association for Computational Linguistics. doi: 10.18653/v1/W18-5448.

Ming Zhu, Aneesh Jain, Karthik Suresh, Roshan Ravindran, Sindhu Tipirneni, and Chandan K. Reddy. XLCoST: A benchmark dataset for cross-lingual code intelligence. *arXiv preprint arXiv:2206.08474*, 2022. doi: 10.48550/arXiv.2206.08474.

# A Pairwise problem overlap in XLCoST

Table 2: Shared problem identifiers between every pair of XLCoST languages, train split. Matched cross-lingual transfer needs a pair to share enough problems to hold out a test fold; only the three bold cells clear 500. No pair outside Python/JavaScript/PHP does, including same-family pairs.

| | C++ | C# | Java | JavaScript | PHP | Python |
|---|---|---|---|---|---|---|
| C++ | — | 115 | 122 | 75 | 17 | 74 |
| C# | 115 | — | 175 | 75 | 14 | 62 |
| Java | 122 | 175 | — | 85 | 11 | 66 |
| JavaScript | 75 | 75 | 85 | — | **1,529** | **2,953** |
| PHP | 17 | 14 | 11 | **1,529** | — | **1,145** |
| Python | 74 | 62 | 66 | **2,953** | **1,145** | — |

# B Full transfer table

Table 3: All eighteen transfer cells for Qwen2.5-Coder-1.5B. *Surface* is the strongest masked-context $n$-gram feature; *name* uses the variable's name alone; *probe* is the residual-stream probe. Majority and shuffled-label controls sit near chance throughout, and $\rho$ is the macro-F1 movement from one test occurrence changing its prediction.

| role | pair | $n$ | surface | name | probe | major. | shuf. | $\rho$ |
|---|---|---|---|---|---|---|---|---|
| accumulator | JavaScript→PHP | 1,976 | 0.951 | 0.897 | 0.904 | 0.361 | 0.470 | 0.0005 |
| accumulator | JavaScript→Python | 3,341 | 0.813 | 0.873 | 0.875 | 0.415 | 0.473 | 0.0004 |
| accumulator | PHP→JavaScript | 1,903 | 0.954 | 0.897 | 0.918 | 0.372 | 0.490 | 0.0005 |
| accumulator | PHP→Python | 1,377 | 0.782 | 0.824 | 0.833 | 0.276 | 0.439 | 0.0008 |
| accumulator | Python→JavaScript | 3,294 | 0.785 | 0.874 | 0.918 | 0.370 | 0.504 | 0.0003 |
| accumulator | Python→PHP | 1,478 | 0.736 | 0.845 | 0.874 | 0.271 | 0.466 | 0.0007 |
| index_key | JavaScript→PHP | 1,976 | 0.940 | 0.834 | 0.897 | 0.384 | 0.478 | 0.0005 |
| index_key | JavaScript→Python | 3,341 | 0.859 | 0.787 | 0.883 | 0.330 | 0.497 | 0.0003 |
| index_key | PHP→JavaScript | 1,903 | 0.964 | 0.838 | 0.880 | 0.376 | 0.516 | 0.0005 |
| index_key | PHP→Python | 1,377 | 0.839 | 0.782 | 0.854 | 0.387 | 0.515 | 0.0008 |
| index_key | Python→JavaScript | 3,294 | 0.871 | 0.822 | 0.915 | 0.314 | 0.470 | 0.0003 |
| index_key | Python→PHP | 1,478 | 0.858 | 0.838 | 0.874 | 0.419 | 0.459 | 0.0008 |
| iterator | JavaScript→PHP | 1,976 | 0.937 | 0.862 | 0.906 | 0.448 | 0.488 | 0.0008 |
| iterator | JavaScript→Python | 3,341 | 0.774 | 0.733 | 0.939 | 0.444 | 0.472 | 0.0005 |
| iterator | PHP→JavaScript | 1,903 | 0.965 | 0.873 | 0.851 | 0.446 | 0.478 | 0.0008 |
| iterator | PHP→Python | 1,377 | 0.576 | 0.788 | 0.865 | 0.428 | 0.462 | 0.0010 |
| iterator | Python→JavaScript | 3,294 | 0.790 | 0.769 | 0.918 | 0.466 | 0.484 | 0.0007 |
| iterator | Python→PHP | 1,478 | 0.741 | 0.788 | 0.902 | 0.475 | 0.438 | 0.0020 |

# C Transfer mechanism

# D Language geometry

**The typing axis is not privileged.** Probing the same activations under every alternative 4-versus-3 partition of the seven languages ($\binom{7}{4} = 35$ groupings minus the real one, so 34 controls) gives

Table 4: Probe transfer per model, averaged within each group. The untrained network shares Qwen2.5-1.5B's architecture and tokenizer with randomly initialised weights; it is the floor a representational claim must clear, and it is itself flat.

| model | close pair | Python pairs | all | vs. untrained |
|---|---|---|---|---|
| Qwen2.5-1.5B | 0.899 | 0.888 | 0.892 | +0.316 |
| Qwen2.5-Coder-1.5B | 0.893 | 0.887 | 0.889 | +0.314 |
| Qwen2.5-1.5B (untrained) | 0.590 | 0.568 | 0.575 | — |

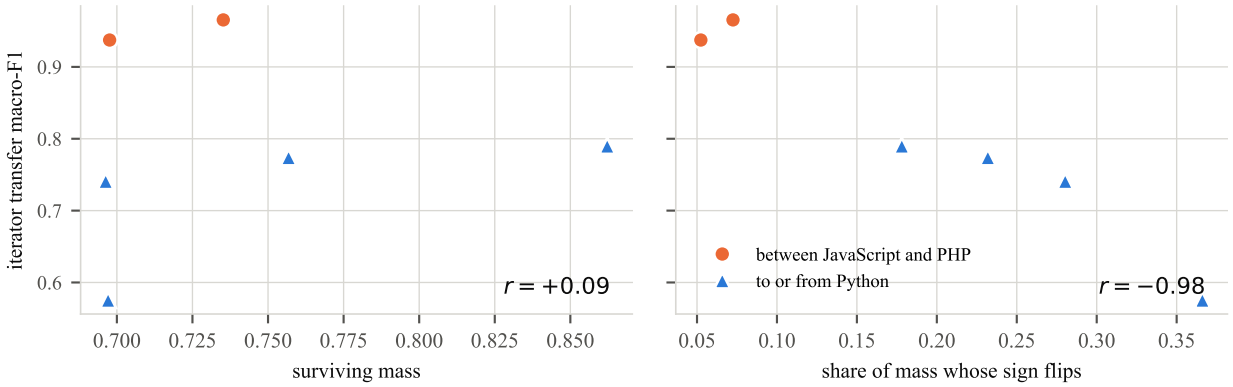


Figure 2: Two candidate explanations for the transfer gap, over the six ordered pairs. Left: how much of the source classifier's discriminative mass the target realises, which is essentially unrelated to transfer. Right: the share of that mass whose sign reverses between a source-fitted and a target-fitted classifier, which tracks it closely. The cues are present in both groups; what changes is where they point.

best-layer macro-F1 0.9959 on average (min 0.9918, max 1.0000), against 1.0000 for the true static/dynamic split; 5 of the 34 controls match or exceed it. Any grouping of these languages is near-perfectly decodable, so the decodability of the typing split is evidence that language identity is encoded, not that type discipline is. At layer 4, where $|r|$ peaks at 0.941, the remaining components carry almost none of the label: PC2 $r = -0.014$, PC3 $r = +0.026$, PC4 $r = +0.171$.

## E Causal interventions

**Scope.** Causal interventions were run for the *boolean* role over 5 languages (C++, Java, JavaScript, PHP, Python), 3 models, and 3 intervention modes (ablate, patch, steer), giving 375 layer-wise measurements. They are reported here rather than in the main text because they were run *within* languages and on a role outside the three this paper transfers, so they cannot support its central claim. Specificity is $|\text{clean} - \text{intervened}|$ divided by the same quantity for a random-position control: how much of the effect is specific to the variable's own token positions rather than to editing anything at all. Two columns are given because the layer with the largest effect is generally not the layer with the best specificity, and quoting either alone misleads.

## F Reproducibility

Every figure is regenerated from committed result files, and a regression test recomputes each of them from those files rather than quoting them, with one exception. The context-pooled untrained row of Table 1 comes from a completed re-run whose per-cell artifacts are pending commitment to `results/lp4fm/masked_probe/`; the span-pooled untrained row is committed, but was measured on the earlier capped sample rather than the three-way intersection the other rows use, so the $+0.269$ margin pairs conditions from two samples and should be read as approximate. Activation extraction is the only step needing an accelerator. The probe is logistic regression on activations mean-pooled over either the occurrence's span or the 16 tokens either side of it excluding the span; the surface baseline

Table 5: Transfer mechanism for the iterator role. *Surviving* is the share of the source classifier's discriminative mass realised in the target; *flip* is the share of that mass whose sign reverses. Surviving mass is uncorrelated with transfer; flipped mass predicts it almost exactly. Source-weighted variants are given for robustness.

| pair | F1 | surviving | flip | agree | flip (src-wt.) |
|---|---|---|---|---|---|
| PHP→JavaScript | 0.965 | 0.735 | 0.072 | +0.895 | 0.072 |
| JavaScript→PHP | 0.937 | 0.698 | 0.052 | +0.945 | 0.068 |
| Python→JavaScript | 0.790 | 0.862 | 0.178 | +0.695 | 0.211 |
| JavaScript→Python | 0.774 | 0.757 | 0.232 | +0.811 | 0.241 |
| Python→PHP | 0.741 | 0.696 | 0.280 | +0.528 | 0.367 |
| PHP→Python | 0.576 | 0.697 | 0.366 | +0.476 | 0.346 |

Table 6: Masking an entire syntactic character class on both sides of the transfer. No class accounts for more than $0.021$ of the $0.39$ gap, so the realignment is distributed rather than carried by block delimiters or statement terminators.

| pair | masked class | macro-F1 | $\Delta$ |
|---|---|---|---|
| PHP→JavaScript | terminator | 0.9496 | -0.0159 |
| PHP→JavaScript | brace | 0.9645 | -0.0010 |
| PHP→JavaScript | bracket | 0.9711 | +0.0057 |
| PHP→JavaScript | paren | 0.9609 | -0.0045 |
| PHP→JavaScript | operator | 0.9441 | -0.0213 |
| PHP→Python | terminator | 0.5786 | +0.0030 |
| PHP→Python | brace | 0.5750 | -0.0006 |
| PHP→Python | bracket | 0.5733 | -0.0023 |
| PHP→Python | paren | 0.5947 | +0.0191 |
| PHP→Python | operator | 0.5801 | +0.0045 |

is a `char_wb` tf-idf vectoriser with $n \in [2, 4]$ followed by logistic regression, fitted on the source language only. Pooling and initialisation are recorded in each store's identity, so a context-pooled store cannot be confused with a span-pooled one downstream.

Table 7: Point-biserial correlation between PC1 of last-token residual activations and a static/dynamic typing label, by layer, with PC1's explained-variance ratio. The sign of $r$ is arbitrary (PCA fixes the axis only up to reflection, and each layer is refit independently); magnitude is what carries meaning.

| layer | $\lvert r\rvert$ | EVR | layer | $\lvert r\rvert$ | EVR |
|---|---|---|---|---|---|
| 0 | 0.883 | 0.481 | 15 | 0.894 | 0.309 |
| 1 | 0.898 | 0.447 | 16 | 0.882 | 0.303 |
| 2 | 0.914 | 0.391 | 17 | 0.874 | 0.296 |
| 3 | 0.939 | 0.373 | 18 | 0.873 | 0.289 |
| 4 | 0.941 | 0.370 | 19 | 0.873 | 0.299 |
| 5 | 0.862 | 0.369 | 20 | 0.903 | 0.291 |
| 6 | 0.828 | 0.361 | 21 | 0.886 | 0.276 |
| 7 | 0.844 | 0.342 | 22 | 0.859 | 0.265 |
| 8 | 0.896 | 0.339 | 23 | 0.842 | 0.252 |
| 9 | 0.919 | 0.333 | 24 | 0.812 | 0.242 |
| 10 | 0.880 | 0.327 | 25 | 0.774 | 0.253 |
| 11 | 0.893 | 0.302 | 26 | 0.694 | 0.255 |
| 12 | 0.917 | 0.292 | 27 | 0.732 | 0.237 |
| 13 | 0.904 | 0.314 | 28 | 0.576 | 0.242 |
| 14 | 0.898 | 0.309 | | | |

Table 8: Causal interventions on boolean-flag occurrences. *Peak-effect spec.* is specificity at the largest-effect layer; *best spec.* is the maximum over layers, with that layer named.

| language | mode | model | peak-effect spec. | best spec. (layer) |
|---|---|---|---|---|
| C++ | ablate | qwen2515b | 4.5× | 31.9× (L4) |
| C++ | ablate | qwen25coder15b | 4.4× | 76.2× (L20) |
| C++ | ablate | starcoder27b | 4.3× | 36.5× (L4) |
| C++ | patch | qwen2515b | 3.6× | 7.9× (L20) |
| C++ | patch | qwen25coder15b | 3.6× | 6.9× (L8) |
| C++ | patch | starcoder27b | 8.1× | 10.4× (L16) |
| C++ | steer | qwen2515b | 45.0× | 169.8× (L4) |
| C++ | steer | qwen25coder15b | 598.5× | 598.5× (L24) |
| C++ | steer | starcoder27b | 6.9× | 341.1× (L4) |
| Java | ablate | qwen2515b | 28.5× | 86.0× (L12) |
| Java | ablate | qwen25coder15b | 12.3× | 62.4× (L12) |
| Java | ablate | starcoder27b | 27.0× | 27.0× (L4) |
| Java | patch | qwen2515b | 6.1× | 7.7× (L12) |
| Java | patch | qwen25coder15b | 5.9× | 7.7× (L12) |
| Java | patch | starcoder27b | 7.5× | 8.6× (L12) |
| Java | steer | qwen2515b | 72.1× | 849.0× (L8) |
| Java | steer | qwen25coder15b | 71.9× | 186.6× (L0) |
| Java | steer | starcoder27b | 252.0× | 252.0× (L24) |
| JavaScript | ablate | qwen2515b | 26.7× | 26.7× (L16) |
| JavaScript | ablate | qwen25coder15b | 28.1× | 116.3× (L24) |
| JavaScript | ablate | starcoder27b | 27.3× | 27.3× (L4) |
| JavaScript | patch | qwen2515b | 6.9× | 6.9× (L8) |
| JavaScript | patch | qwen25coder15b | 6.6× | 7.4× (L12) |
| JavaScript | patch | starcoder27b | 8.1× | 8.2× (L16) |
| JavaScript | steer | qwen2515b | 24.5× | 76.4× (L8) |
| JavaScript | steer | qwen25coder15b | 65.2× | 149.5× (L4) |
| JavaScript | steer | starcoder27b | 11.2× | 60.8× (L28) |
| PHP | ablate | qwen2515b | 15.0× | 102.4× (L20) |
| PHP | ablate | qwen25coder15b | 26.5× | 79.2× (L20) |
| PHP | ablate | starcoder27b | 19.4× | 19.7× (L8) |
| PHP | patch | qwen2515b | 7.2× | 7.2× (L4) |
| PHP | patch | qwen25coder15b | 4.7× | 7.4× (L0) |
| PHP | patch | starcoder27b | 6.3× | 9.3× (L16) |
| PHP | steer | qwen2515b | 190.3× | 190.3× (L24) |
| PHP | steer | qwen25coder15b | 11.5× | 134.0× (L8) |
| PHP | steer | starcoder27b | 8.7× | 100.1× (L4) |
| Python | ablate | qwen2515b | 5.8× | 36.7× (L24) |
| Python | ablate | qwen25coder15b | 6.8× | 33.1× (L16) |
| Python | ablate | starcoder27b | 26.5× | 26.5× (L4) |
| Python | patch | qwen2515b | 6.0× | 7.2× (L16) |
| Python | patch | qwen25coder15b | 6.7× | 7.8× (L16) |
| Python | patch | starcoder27b | 7.9× | 9.7× (L12) |
| Python | steer | qwen2515b | 209.1× | 827.9× (L16) |
| Python | steer | qwen25coder15b | 1025.8× | 1025.8× (L24) |
| Python | steer | starcoder27b | 8.6× | 55.5× (L4) |

# NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

**The checklist answers are an integral part of your paper submission.** They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Section 1 state the transfer figures, the untrained floor, and the mechanism result, and each is the number reported in Sections 4 and 5. Claims are scoped to three languages, three roles, and two trained models throughout.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

Justification: The Limitations paragraph of Section 6 covers the two-point typological axis, the single model family and scale, the extractor-definition difference affecting the iterator role, the scope of the mechanism analysis, and the absence of causal evidence on the cross-lingual cells.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper reports no theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 2 gives the corpus, the matching procedure, the occurrence cap, the split policy, the baseline feature families, and the probe's layer-selection rule. Appendix F names the scripts, the model identifiers, and the classifier settings.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: All result tables are committed as CSVs under `results/lp4fm/`, and the analysis scripts are released. Appendix F gives the commands. An anonymised mirror is provided for the review period.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.

- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 2 specifies the cap, the frozen hash split, and the source-validation layer selection. Appendix F gives the $n$-gram vectoriser configuration and the logistic-regression settings.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

   Justification: We report resolution ($\rho$, the macro-F1 movement from one test occurrence changing its prediction) for every cell rather than confidence intervals, and state in Section 2 which comparisons clear it and which do not. Per-cell cluster-bootstrap intervals are computed by the released code but are not reported in this version; the mechanism correlations rest on six ordered pairs and are reported with intervals in Section 5.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
   - If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Appendix F reports the accelerator used and the wall-clock cost of activation extraction, which dominates. The surface baselines and the mechanism analyses are CPU-only.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work analyses publicly released models on a public benchmark of competitive-programming solutions. It involves no human subjects, no personally identifying data, and no deployment.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: The work is an analysis of how existing code models represent variable roles. We identify no societal impact specific to it beyond that of the public models and benchmark it studies.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper releases no data or models. It analyses publicly available ones.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: XLCoST is cited and its licence respected. The two Qwen model releases are cited by name and identifier in Section 2 and Appendix F.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

Justification: The released analysis scripts and result files are documented in Appendix F, including how each table is regenerated from the committed CSVs.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper involves no crowdsourcing and no human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper involves no human subjects, so no IRB review was required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [No]

Justification: Large language models are the object of study, not a component of the method. They were used for writing and editing assistance only, which the question exempts from declaration.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.